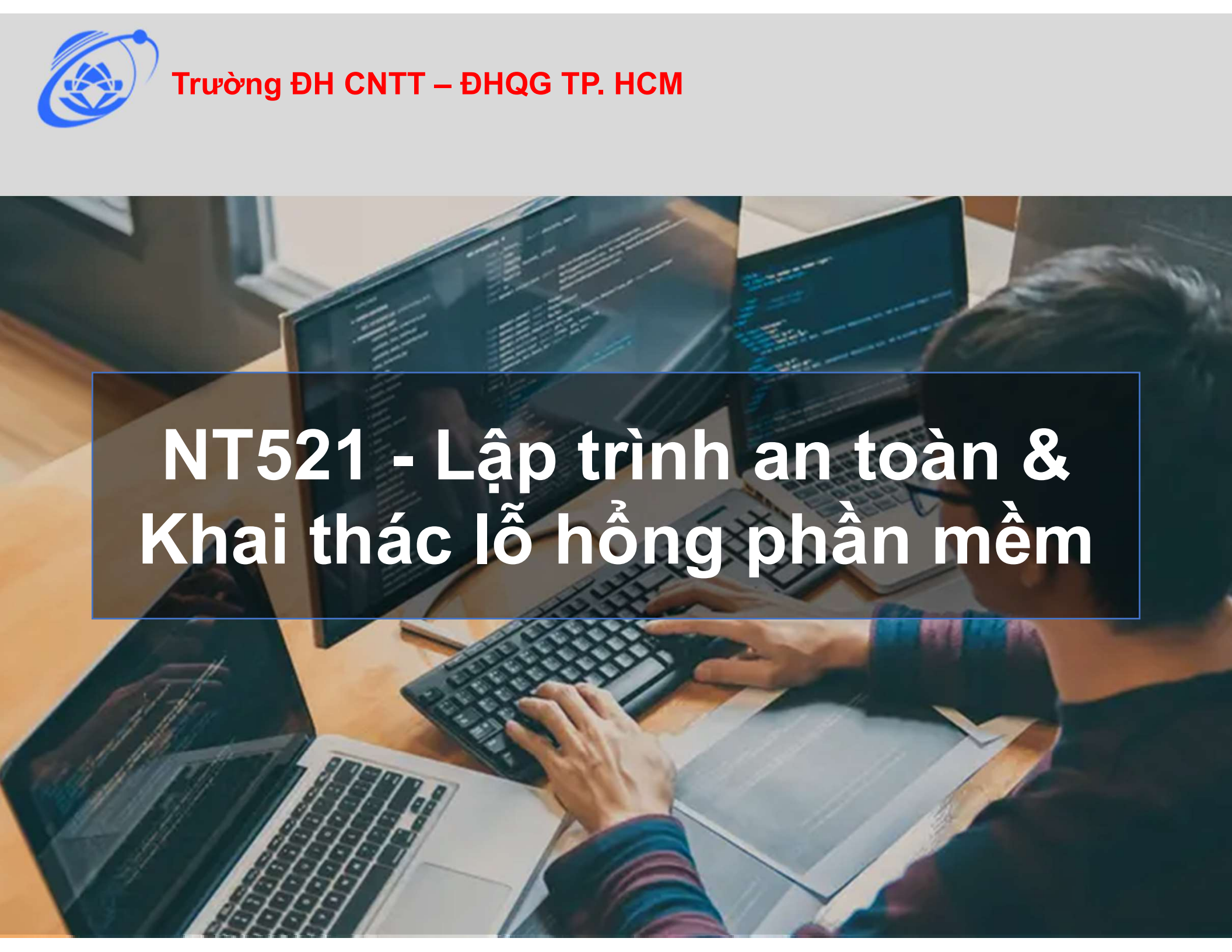




Trường ĐH CNTT – ĐHQG TP. HCM



NT521 - Lập trình an toàn & Khai thác lỗ hổng phần mềm

Lỗi hỏng Chuỗi định dạng & Tấn công ROP



- Lỗi hỏng Chuỗi định dạng (Format String)
- Tấn công ROP (Return Oriented Programming)

Lỗi hỏng Chuỗi định dạng

Format String Vulnerability

Khai thác thực tế Format String



Search Results

There are **751** CVE Records that match your search.

Name	
CVE-2022-34747	A format string vulnerability in Zyxel NAS326 firmware versions prior to V5.21(AAZF.12)C0 could allow an attacker to achieve una
CVE-2022-31753	The voice wakeup module has a vulnerability of using externally-controlled format strings. Successful exploitation of this vulnerab
CVE-2022-27177	A Python format string issue leading to information disclosure and potentially remote code execution in ConsoleMe for all versions
CVE-2022-26674	ASUS RT-AX88U has a Format String vulnerability, which allows an unauthenticated remote attacker to write to arbitrary memory
CVE-2022-2652	Depending on the way the format strings in the card label are crafted it's possible to leak kernel stack memory. There is also the j
CVE-2022-26393	The Baxter Spectrum WBM is susceptible to format string attacks via application messaging. An attacker could use this to read m
CVE-2022-26392	The Baxter Spectrum WBM (v16, v16D38) and Baxter Spectrum WBM (v17, v17D19, v20D29 to v20D32) when in superuser mod
CVE-2022-24051	MariaDB CONNECT Storage Engine Format String Privilege Escalation Vulnerability. This vulnerability allows local attackers to esca
CVE-2022-22299	A format string vulnerability [CWE-134] in the command line interpreter of FortiADC version 6.0.0 through 6.0.4, FortiADC version
CVE-2022-1215	A format string vulnerability was found in libinput
CVE-2021-44532	Node.js < 12.22.9, < 14.18.3, < 16.13.2, and < 17.3.1 converts SANs (Subject Alternative Names) to a string format. It uses thi
CVE-2021-43041	An issue was discovered in Kaseya Unitrends Backup Appliance before 10.5.5. A crafted HTTP request could induce a format string
CVE-2021-42911	A Format String vulnerability exists in DrayTek Vigor 2960 <= 1.5.1.3, DrayTek Vigor 3900 <= 1.5.1.3, and DrayTek Vigor 300B ·
CVE-2021-41193	wire-avs is the audio visual signaling (AVS) component of Wire, an open-source messenger. A remote format string vulnerability i
CVE-2021-35331	** DISPUTED ** In Tcl 8.6.11, a format string vulnerability in nmakehlp.c might allow code execution via a crafted file. NOTE: mu
CVE-2021-33535	In Weidmueller Industrial WLAN devices in multiple versions an exploitable format string vulnerability exists in the iw_console cor

MITRE CVE relating to Format String:

<https://cve.mitre.org/cgi-bin/cvekey.cgi?keyword=format+string>



Khai thác thực tế Format String



High Severity

Security Bulletin: A format string security vulnerability has been identified in IBM Spectrum Scale (CVE-2021-29740)

May 31, 2021

Categorized High Severity

Share this post:



A security vulnerability has been identified in IBM Spectrum Scale file system that could allow an attacker to execute arbitrary code. A fix for this vulnerability is available.

CVE(s): [CVE-2021-29740](#)

Affected product(s) and affected version(s):

Affected Product(s)	Version(s)
IBM Spectrum Scale	5.0.0 – 5.0.5.6
IBM Spectrum Scale	5.1.0 – 5.1.0.3

Refer to the following reference URLs for remediation and additional vulnerability details:

Source Bulletin: <https://www.ibm.com/support/pages/node/6457629>

X-Force Database: <https://exchange.xforce.ibmcloud.com/vulnerabilities/201474>

IBM Spectrum Scale:

<https://www.ibm.com/blogs/psirt/security-bulletin-a-format-string-security-vulnerability-has-been-identified-in-ibm-spectrum-scale-cve-2021-29740/>



Khai thác thực tế

Format String



CVE-2022-1215 Detail

Current Description

A format string vulnerability was found in libinput

[View Analysis Description](#)

Severity

CVSS Version 3.x

CVSS Version 2.0

CVSS 3.x Severity and Metrics:

NIST: NVD

Base Score: **7.8 HIGH**

Vector: CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

NVD Analysts use publicly available information to associate vector strings and CVSS scores. We also display any CVSS information provided within the CVE List from the CNA.

Note: NVD Analysts have published a CVSS score for this CVE based on publicly available information at the time of analysis. The CNA has not provided a score within the CVE List.

QUICK INFO

CVE Dictionary Entry:
CVE-2022-1215

NVD Published Date:
06/02/2022

NVD Last Modified:
06/09/2022

Source:
Red Hat, Inc.

References to Advisories, Solutions, and Tools

By selecting these links, you will be leaving NIST webspace. We have provided these links to other web sites because they may have information that would be of interest to you. No inferences should be drawn on account of other sites being referenced, or not, from this page. There may be other web sites that are more appropriate for your purpose. NIST does not necessarily endorse the views expressed, or concur with the facts presented on these sites. Further, NIST does not endorse any commercial products that may be mentioned on these sites. Please address comments about this page to nvd@nist.gov.

Hyperlink	Resource
https://seclists.org/oss-sec/2022/q2/47	Mailing List Third Party Advisory

Weakness Enumeration

CWE-ID	CWE Name	Source
CWE-134	Use of Externally-Controlled Format String	NIST Red Hat, Inc.

Known Affected Software Configurations [Switch to CPE 2.2](#)

Configuration 1 [\(hide\)](#)

cpe:2.3:a:freedesktop:libinput:*:*:*:*:*:* Show Matching CPE(s)	From (including) 1.10.0	Up to (excluding) 1.18.2
cpe:2.3:a:freedesktop:libinput:*:*:*:*:*:* Show Matching CPE(s)	From (including) 1.19.0	Up to (excluding) 1.19.4

CVE-2022-1215 was found in libinput:
<https://nvd.nist.gov/vuln/detail/CVE-2022-1215>



Chuỗi định dạng

Chuỗi định dạng là gì?



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int
5 main(int argc, char *argv[])
6 {
7     char *format = "%s";
8     char *arg1 = "Hello World!\n";
9     printf(format, arg1);
10    return EXIT_SUCCESS;
11 }
```

A red arrow originates from the right side of the slide and points directly to the format string "%s" in line 7 of the code.

Chuỗi định dạng là gì?



- Nhiều hàm in có sử dụng chuỗi định dạng

```
#include <stdio.h>

int printf(const char *format, ...);
int fprintf(FILE *stream, const char *format, ...);
int sprintf(char *str, const char *format, ...);
int snprintf(char *str, size_t size, const char *format, ...);

#include <stdarg.h>

int vprintf(const char *format, va_list ap);
int vfprintf(FILE *stream, const char *format, va_list ap);
int vsprintf(char *str, const char *format, va_list ap);
int vsnprintf(char *str, size_t size, const char *format, va_list ap);
```



- Format string – Chuỗi định dạng: một chuỗi chứa các ký hiệu chuyển đổi để định dạng cho dữ liệu

Ký hiệu	Kiểu định dạng
d	Số nguyên (integer) - 4 bytes
u	Số nguyên không dấu – 4 bytes
x	Dạng hexan – 4 bytes
s	Kiểu chuỗi (string) – con trỏ
c	Ký tự - 1 byte

- Một số ký hiệu xác định độ dài

Ký hiệu	Độ dài	Kiểu định dạng
hh	1 byte	Ký tự
h	2 byte	Short int
l	4 byte	Long int
ll	8 byte	Long long int

Ví dụ: “%hd”

- Định dạng đầu ra

```
printf("%03d.%03d.%03d.%03d", 127, 0, 0, 1);  
    127.000.000.001  
printf("%.2f", 5.6732);  
    5.67  
printf("%#010x", 3735928559);  
    0xdeadbeef
```

- Đếm số byte

```
printf("%s%n", "01234", &n);  
    n = 5
```

%n: in ra độ dài của chuỗi vừa được in ra trước đó

Lỗi hỏng format string?

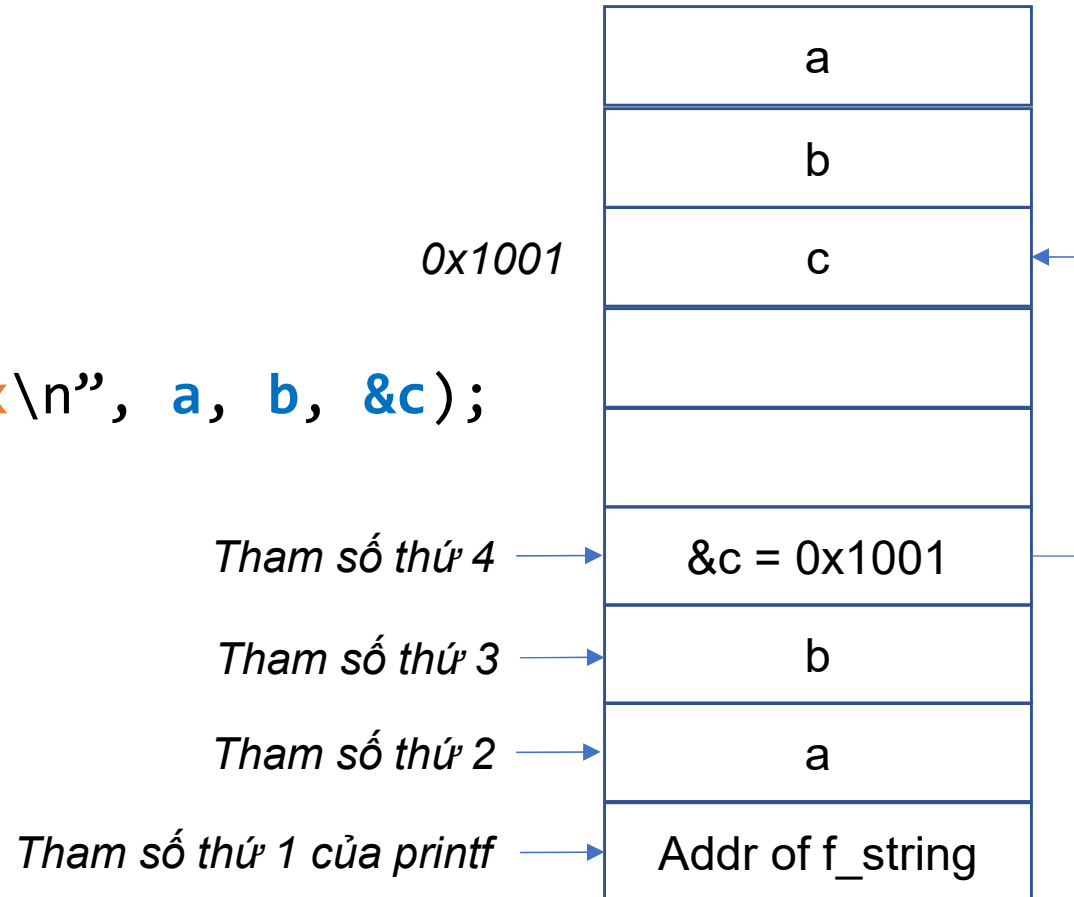
• Đoạn code:

```
int a = 5;
int b = 10;
int c = 15;
printf("a=%d, b=%d, c at %08x\n", a, b, &c);
```

Kết quả in ra gì?

a=5, b=10, c at 00001001

→ So easy!



Với **printf**, chuỗi format là **tham số thứ nhất**, các ký hiệu trong đó định dạng lần lượt cho các **tham số 2, 3, 4...** của printf

Ví dụ: **2 ký hiệu %d** định dạng tham số thứ 2 và 3 của printf là a và b, **%08x** định dạng tham số thứ 4 là địa chỉ của c

Lỗi hỏng format string?

- Vậy với đoạn code:

```
int a = 5;
```

```
int b = 10;
```

```
int c = 15;
```

```
printf("a=%d, b=%d, c at %08x\n", a, b);
```

→ Không có tham số **&c**

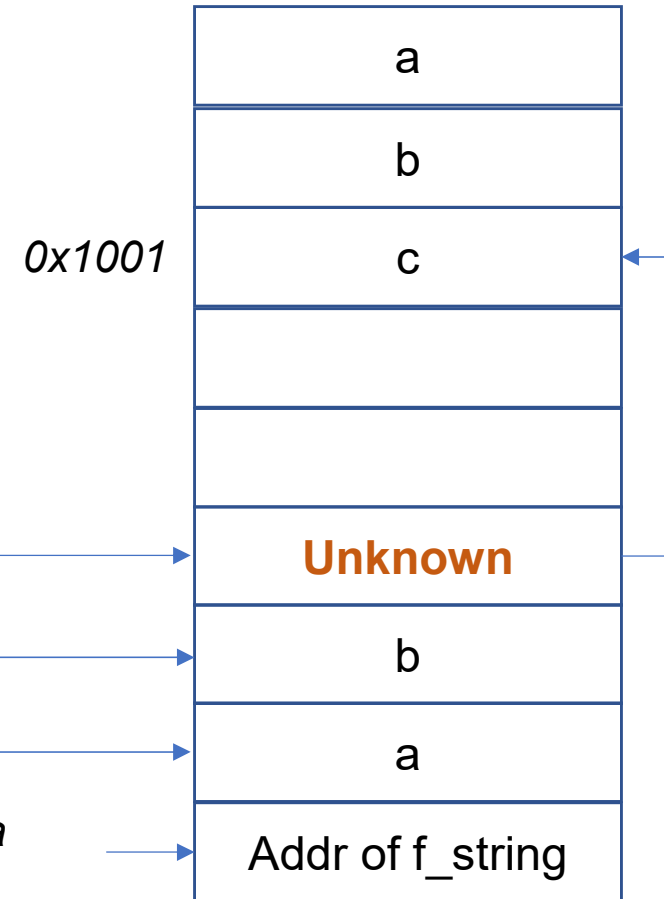
- Ở vị trí tham số thứ 4 là giá trị không xác định.
- Mặc dù vậy, dựa vào chuỗi format, **printf vẫn xem vị trí đó là tham số thứ 4 của nó và in giá trị tại ô nhớ đó**
- Lượng dữ liệu in/ghi ra phụ thuộc vào số ký hiệu trong chuỗi format

Tham số thứ 4

Tham số thứ 3

Tham số thứ 2

Tham số thứ 1 của printf



Call printf

Kết quả in ra gì?

```
a=5, b=10, c at <unknown>
```

Lỗi hỏng format string?

- User có thể điều khiển chuỗi định dạng

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int
5 main(int argc, char *argv[])
6 {
7     char buf[100];
8     fgets(buf, 100, stdin);
9     printf(buf);
10    return EXIT_SUCCESS;
11 }
```

Vấn đề ở đây là gì?

- Printf ở dòng 9 không có chuỗi định dạng, chỉ có tham số là chuỗi của người dùng
→ nếu cố tình nhập buf có dạng format string, nó sẽ được xem là format string của printf và in dữ liệu ra tương ứng
→ Hãy thử nhập các format string khác nhau!

Lỗi hỏng format string

Crash chương trình



- Chuỗi 1: “%s%s%s%s%s...”
 - `printf(“%s%s%s%s%s...”)`
 - Với mỗi %s, printf lấy ra 1 giá trị từ trong stack, xem đó là 1 địa chỉ và in ra nội dung đang lưu tại địa chỉ đó (cho đến khi gặp ký tự NULL).
 - Giá trị lấy ra từ stack với %s có thể không phải là 1 địa chỉ → việc xem là 1 địa chỉ và truy xuất có thể gây lỗi → crash chương trình.
 - Hoặc là 1 địa chỉ không cho phép truy xuất → crash.

```
ubuntu@ubuntu: ~/NT521/formatstring
ubuntu@ubuntu:~/NT521/formatstring$ python -c 'print("%s%s%s")' | ./format
Segmentation fault (core dumped)
ubuntu@ubuntu:~/NT521/formatstring$
```



Lỗi hỏng format string

Đọc dữ liệu trên stack



- Chuỗi 2: “**AAAA**” + “**%08x.**”*10
 - 4 ký tự A
 - 10 chuỗi định dạng “%08x.”

```
ubuntu@ubuntu: ~/NT521/formatstring
ubuntu@ubuntu:~/NT521/formatstring$ python -c 'print("AAAA"+"%08x."*10)' | ./format
AAAA00000064.f7fb35a0.00f0b5ff.ffffcfbe.00000001.000000c2.ffffd0b4.ffffcfbe.ffffd0bc.41414141.
ubuntu@ubuntu:~/NT521/formatstring$
```

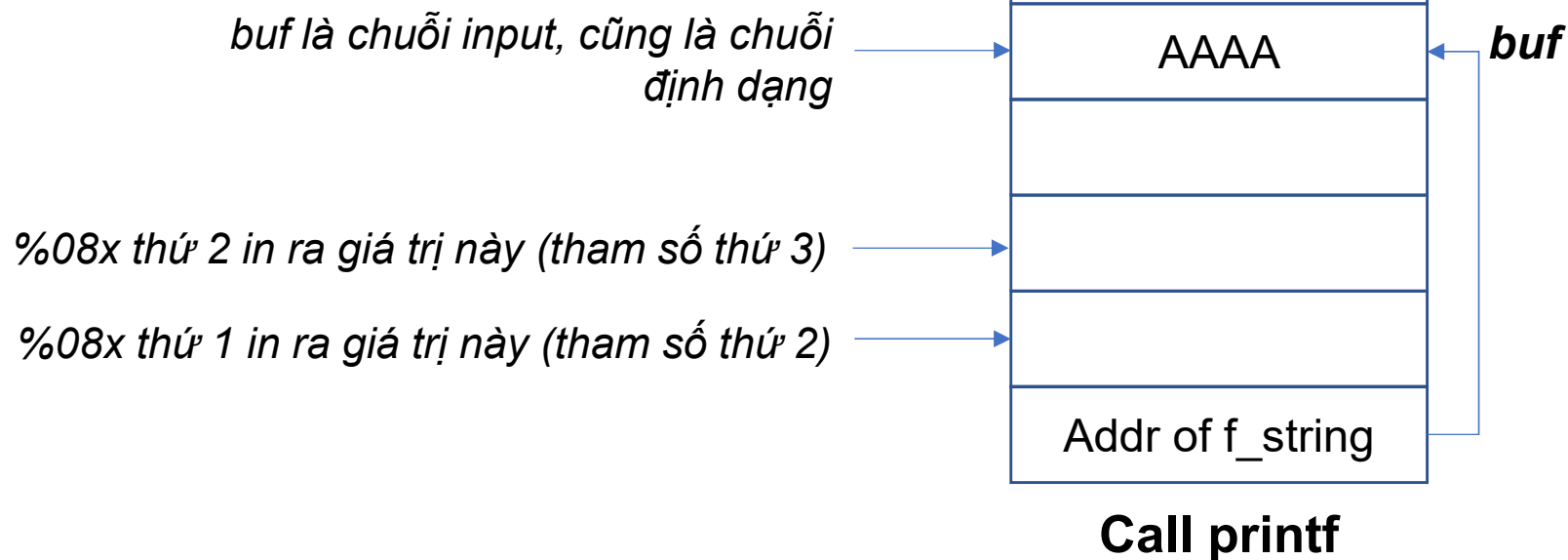
- `printf("AAAA%08x.%08x.%08x...")`
 - Chuỗi định dạng được người dùng nhập, do đó nằm trên stack
 - Đọc dữ liệu trong stack
 - 10 chuỗi “%08x” sẽ đọc được 10 word (40 bytes)
 - Kích thước đầu vào bị giới hạn
 - *Tối đa đọc được bao nhiêu bytes?*



Lỗi hỏng format string

Đọc dữ liệu trên stack

- `printf("AAAA%08x.%08x.%08x...")`
 - Chuỗi định dạng được người dùng nhập, do đó nằm trên stack
 - Đọc dữ liệu trong stack
 - 10 chuỗi "`%08x`" sẽ đọc được 40 bytes
 - Kích thước đầu vào bị giới hạn
 - *Tối đa đọc được bao nhiêu bytes?*



Lỗi hỏng format string

Đọc dữ liệu tại 1 địa chỉ

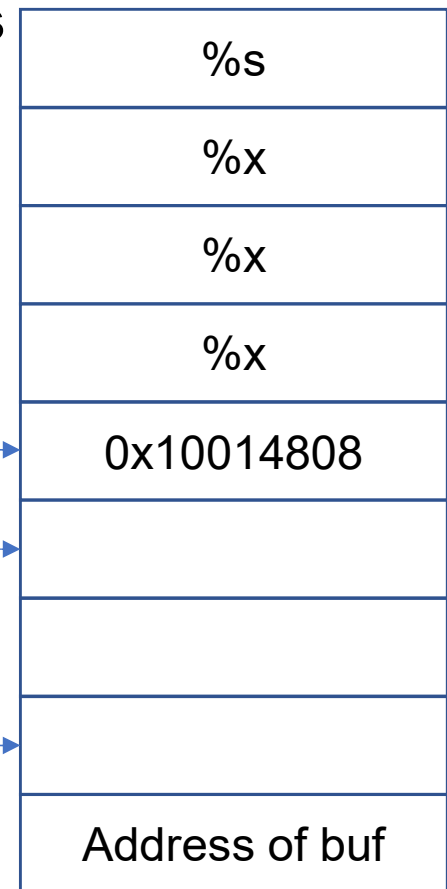
- Chuỗi 3: “<address> %x %x %x... %s”
 - Trong chuỗi format string có địa chỉ cần đọc dữ liệu
 - Cần 1 số lượng *n* chuỗi %x trung gian
 - %s là format dành cho việc đọc dữ liệu tại **address**
 - Để xác định *n*, cần xem address sẽ là tham số thứ mấy của printf?

```
ubuntu@ubuntu: ~/NT521/formatstring
ubuntu@ubuntu:~/NT521/formatstring$ python -c 'print("\x54\x81\x04\x08"+"%x."*9+"%s")' | ./format
T064.f7fb35a0.f0b5ff.ffffcfbe.1.c2.ffffd0b4.ffffcfbe.ffffd0bc./lib/ld-linux.so.2
ubuntu@ubuntu:~/NT521/formatstring$
```

%s in ra nội dung tại địa chỉ này

%x thứ 3 in ra giá trị này

%x đầu tiên in ra giá trị này



buf

Call printf

Lỗi hỏng format string

Truy xuất tham số trực tiếp



- Cú pháp: `printf(“%<arg#>$<format>”)`
 - Ví dụ: `printf(“%3$d”, 1, 2, 3) → Kết quả: “3”`

```
[Slate][MBE]$ for i in {10..100}; do echo "%$i" '$s' | ./a.out arg1 arg2;done
__libc_start_main
./a.out
arg1
arg2
(null)
GREP_COLOR=1;32
XDG_VTNR=2
XDG_SESSION_ID=c1
SHELL=/bin/zsh
ZSH=/home/branden/.oh-my-zsh
USER=branden
PAGER=less
MOZ_PLUGIN_PATH=/usr/lib/mozilla/plugins
LSCOLORS=Gxfxcxdxbxegedabagacad
PATH=/usr/local/sbin:/usr/local/bin:/usr/bin:/usr/b
/bin
lea     eax, [ebp+arg_0]
push    eax
mov     esi, 100h
push    esi
push    [ebp+arg_4]
push    edi
call    sub_314623
test    eax, eax
ja      short loc_31306D
cmp     [ebp+arg_0], esi
jz      short loc_31308F
; CODE XREF: sub_311
; sub_312FD8+55
push    0Dh
call    sub_31411B
; CODE XREF: sub_311
; sub_312FD8+49
call    sub_3140F3
test    eax, eax
jg      short loc_31307D
call    sub_3140F3
jmp     short loc_31308C
; CODE XREF: sub_311
call    sub_3140F3
```





- Chuỗi 3 (nâng cấp): “<address>%k\$s”
 - Trong chuỗi format string có địa chỉ cần đọc dữ liệu
 - Thay vì cần *n* chuỗi %x trung gian, sử dụng ký hiệu truy xuất trực tiếp
 - %s là format dành cho việc đọc dữ liệu tại **address**
 - Để xác định **k**, cần xem address sẽ là tham số thứ mấy của printf?

```
ubuntu@ubuntu: ~/NT521/formatstring
ubuntu@ubuntu:~/NT521/formatstring$ python -c 'print("\x54\x81\x04\x08%10$s")' | ./format
T/lib/ld-linux.so.2
ubuntu@ubuntu:~/NT521/formatstring$
```

%s in ra nội dung tại địa chỉ này

%x thứ 3 in ra giá trị này

%x đầu tiên in ra giá trị này

%s
%x
%x
%x
0x10014808
Address of buf

buf

Call printf



- Sử dụng %n
 - Tham số là 1 biến con trỏ (pointer)
 - Ghi số lượng byte đã được in ra trước đó

```
[Slate][MBE]$ python -c 'print("AAAA"+"%x."*5+"%x%x")' | ./a.out
AAAA64.f76f2600.f75d16b5.41414141.252e7825.78252e782e78252e
[Slate][MBE]$ python -c 'print("AAAA"+"%x."*2+"%x%x")' | ./a.out
AAAA64.f779d600.f767c6b541414141
[Slate][MBE]$ python -c 'print("AAAA"+"%x."*2+"%x%n")' | ./a.out
[1] 10107 done python -c 'print("AAAA"+"%x."*2+
"%x%n")' |
10108 segmentation fault (core dumped) ./a.out
[Slate][MBE]$
```

- Ví dụ trên:
 - Lệnh python 2: %x cuối cùng in ra giá trị 0x41414141
 - Lệnh python 3: %x cuối được thay bằng %n, thay vì in ra 0x41414141, printf xem đó là 1 địa chỉ để ghi dữ liệu → crash

Biện pháp phòng tránh



- Xác định cụ thể chuỗi định dạng được dùng trong chương trình, không dựa vào input người dùng nhập vào.
- Cung cấp đủ các tham số cho printf kèm theo chuỗi định dạng khi cần in dữ liệu.
- Chú ý các cảnh báo của compiler.
- Address randomization.
- ...



Tấn công ROP

(Return Oriented Programming)

Tổng quan về buffer overflow



- Khai thác lỗ hổng **tràn bộ đệm - buffer overflow**:
 - Trường hợp lý tưởng:
 - Có khả năng kiểm soát return addr: không dùng stack canary, có thể ghi đè đến vị trí return addr
 - Có thể truyền shellcode vào stack và thực thi: -z execstack
 - Các trường hợp ràng buộc hơn?
 - **ROP (Return Oriented Programming)**
 - *Stack không cho phép thực thi*
 - **Off-by-one**
 - *Giới hạn kích thước buffer có thể bị ghi đè, không đủ để ghi đè return addr*
 - **Return-to-libc**
 - *Shellcode cần viết quá phức tạp hoặc quá dài, stack không cho phép thực thi code*



Tấn công ROP: Vì sao?

- Tấn công tràn bộ đệm trên stack trong trường hợp KHÔNG được phép thực thi code trên stack.

```
gdb-peda$ checksec
CANARY      : disabled
FORTIFY     : disabled
NX          : ENABLED
```



- ***Non-executable stack - DEP***
- Biên dịch với ***-z noexecstack*** (mặc định)

Tấn công ROP: Vì sao? (2)



- **DEP – Data Execution Prevention**

- Một kỹ thuật giảm thiểu tấn công bằng cách đảm bảo chỉ có **section chứa code** mới được gán nhãn là **thực thi được**.
- Giảm thiểu code injection hay payload chứa shellcode.
- Một số ký hiệu: DEP, NX, W^X.

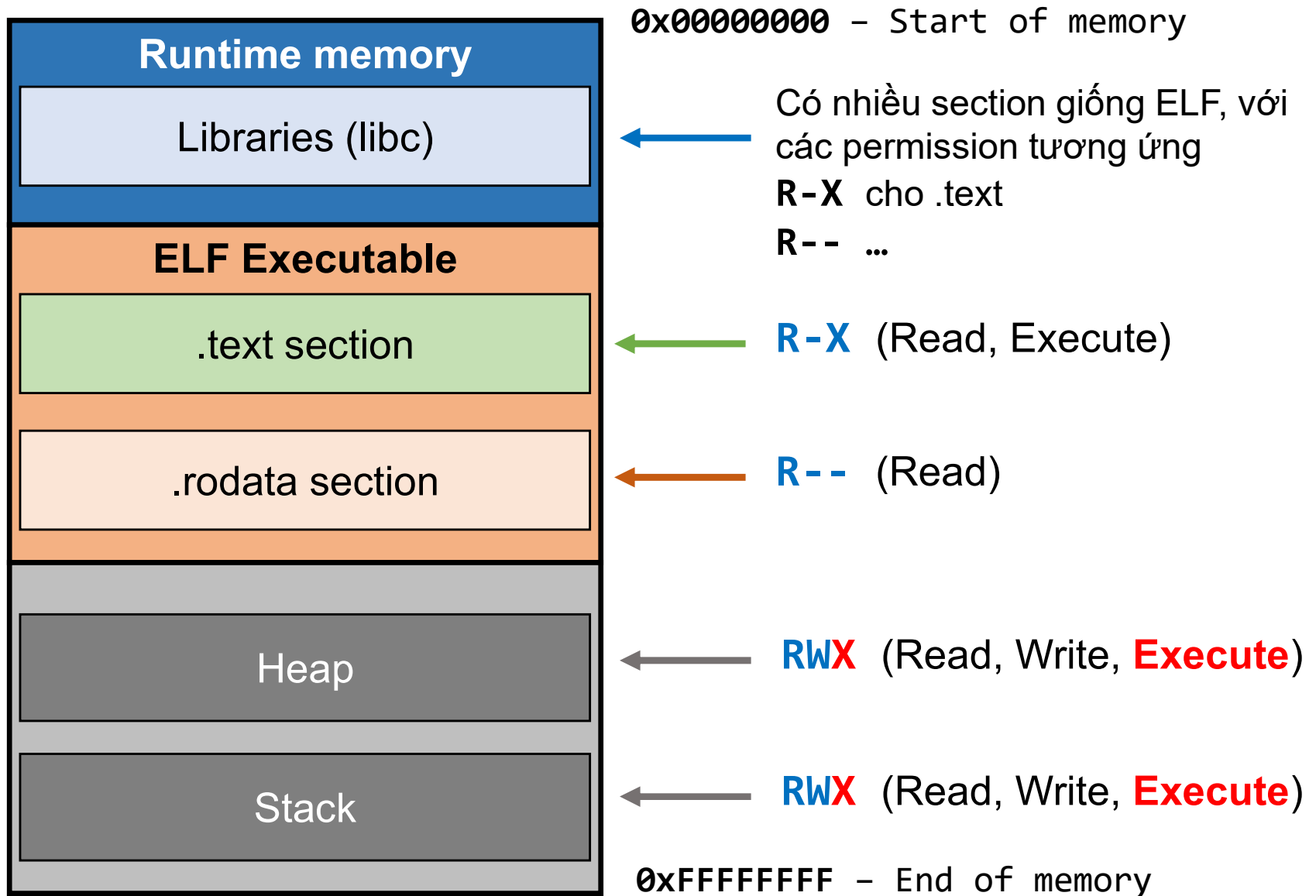
```
gdb-peda$ checksec
CANARY      : disabled
FORTIFY     : disabled
NX          : ENABLED
```

- Cơ bản về DEP:
 - **Không có** section/segment bộ nhớ nào được phép **cùng lúc** cho phép **Ghi và Thực thi: W^X**.
 - Các section dữ liệu (data): stack, heap, .bss, .ro, .data
 - Các section chứa code: .text, .plt



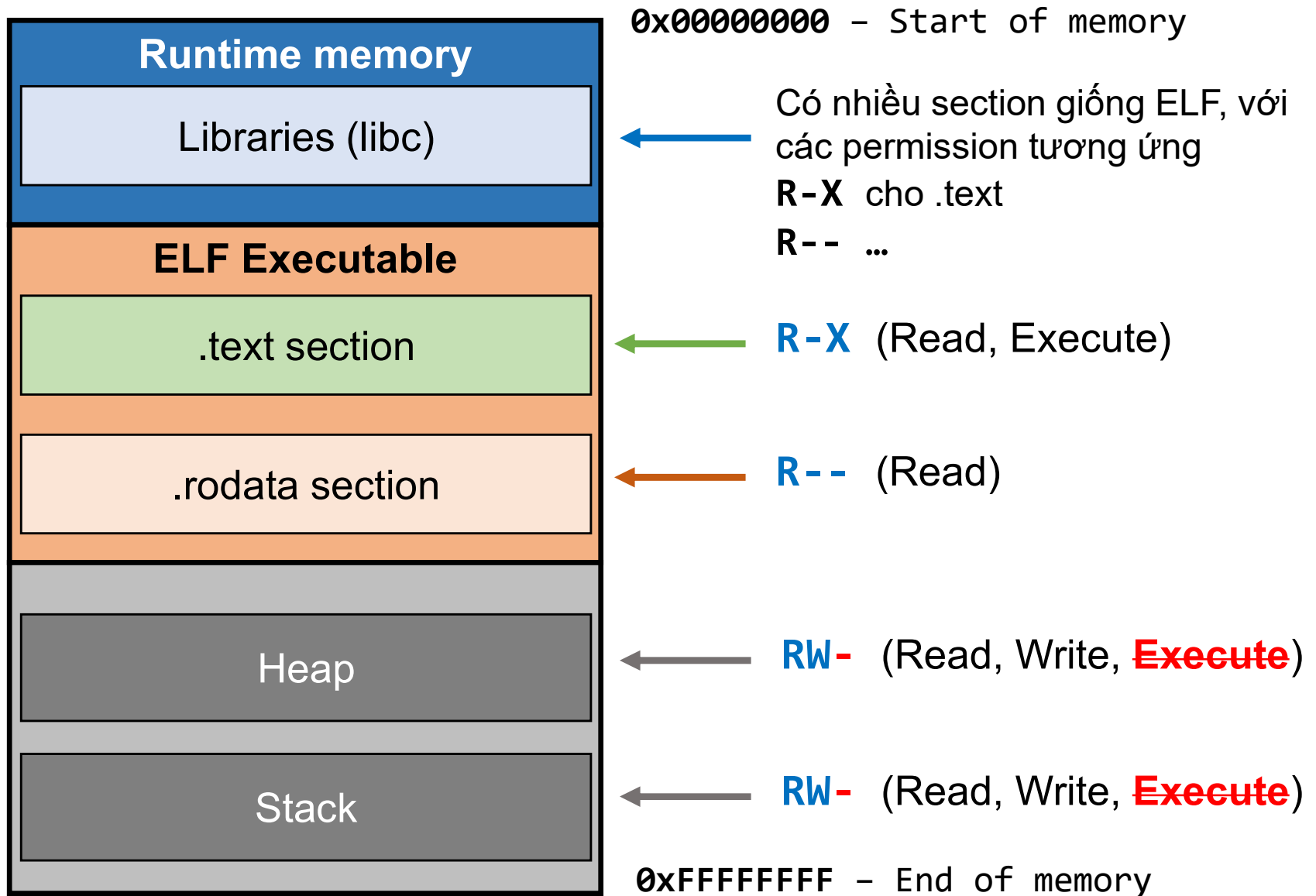
Tấn công ROP: Vì sao? (3)

- Nếu không dùng **DEP**?



Tấn công ROP: Vì sao? (4)

- Nếu dùng **DEP**?



Tấn công ROP: Vì sao? (5)



- Truyền shellcode vào stack để thực thi khi có sử dụng DEP?

→ **Segmentation Fault**



Tấn công ROP: Bypass DEP



- Ý tưởng:
 - Tận dụng code có sẵn trong chương trình.
- **Vì sao không tấn công return-to-libc?**
 - Cần có hàm *libc*
 - Thực thi các hàm *libc* có thể có tác dụng phụ
 - Address randomization có thể khiến các địa chỉ hàm *libc* khó đoán.
- **ROP – Return Oriented Programming**
 - Điểm khác biệt: không cần dùng toàn bộ hàm, chỉ sử dụng các đoạn code “đặc biệt” có các lệnh cần thiết – **gadget**.
 - Có thể xem là các *hàm nhỏ*
 - Luôn kết thúc bằng lệnh **ret**
 - Nếu có thể tìm đủ các *hàm nhỏ* như vậy, attacker có thể thực hiện hành vi mong muốn.



Tấn công ROP: Gadgets



- **Gadgets**

- là một chuỗi liên tục các lệnh assembly có ý nghĩa.
- kết thúc bằng lệnh **ret**.

```
pop eax  
ret
```

```
mov [ecx], ebx  
ret
```

```
mov edx, eax  
mov ebx, ecx  
add eax, 20  
ret
```

```
lea [esp + 12]  
push edx  
ret
```

```
mov [ecx], 1  
ret
```

- Nhiều gadgets được nối (chain) với nhau để tạo thành 1 chuỗi lệnh hiện thực hành vi của attacker tương tự như shellcode.
 - **ROP Chain**

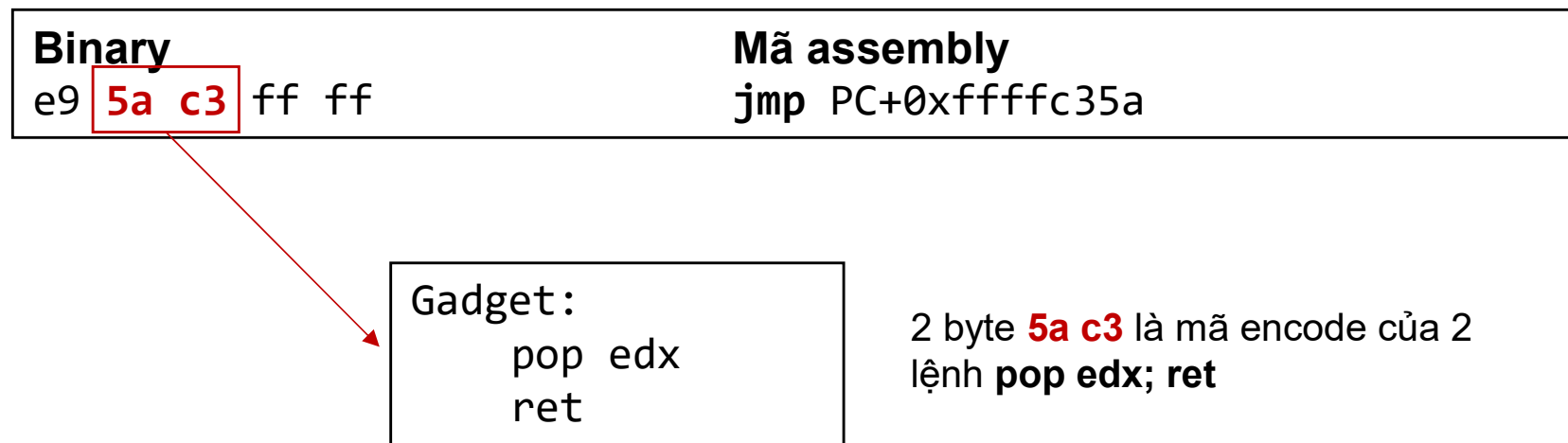


Tấn công ROP: Gadgets



- **Tìm gadget như thế nào?**

- Gadget có thể là các lệnh trong các hàm.
- Gadget có thể được giấu bên trong các byte của các lệnh assembly khác.



- ROPgadget

```
$ ./ROPgadget ./smashme.bin -att -asm "popl %edx; ret"  
  
Gadgets information  
=====
```

0x0806eb0a:	"\x5a\xc3 <==> popl %edx; ret"
0x08081736:	"\x5a\xc3 <==> popl %edx; ret"

```
Total opcodes found: 2
```



Tấn công ROP: ROP chain

- ROP chain hoạt động như thế nào?
 - Làm sao để chain các gadget?

Shellcode: `exit(0)`

```
xor eax, eax
xor ebx, ebx
inc eax
int 0x80
```



gadget 1

```
xor eax, eax
ret
```



gadget 2

```
xor ebx, ebx
ret
```



gadget 3

```
inc eax
ret
```



```
int 0x80
```

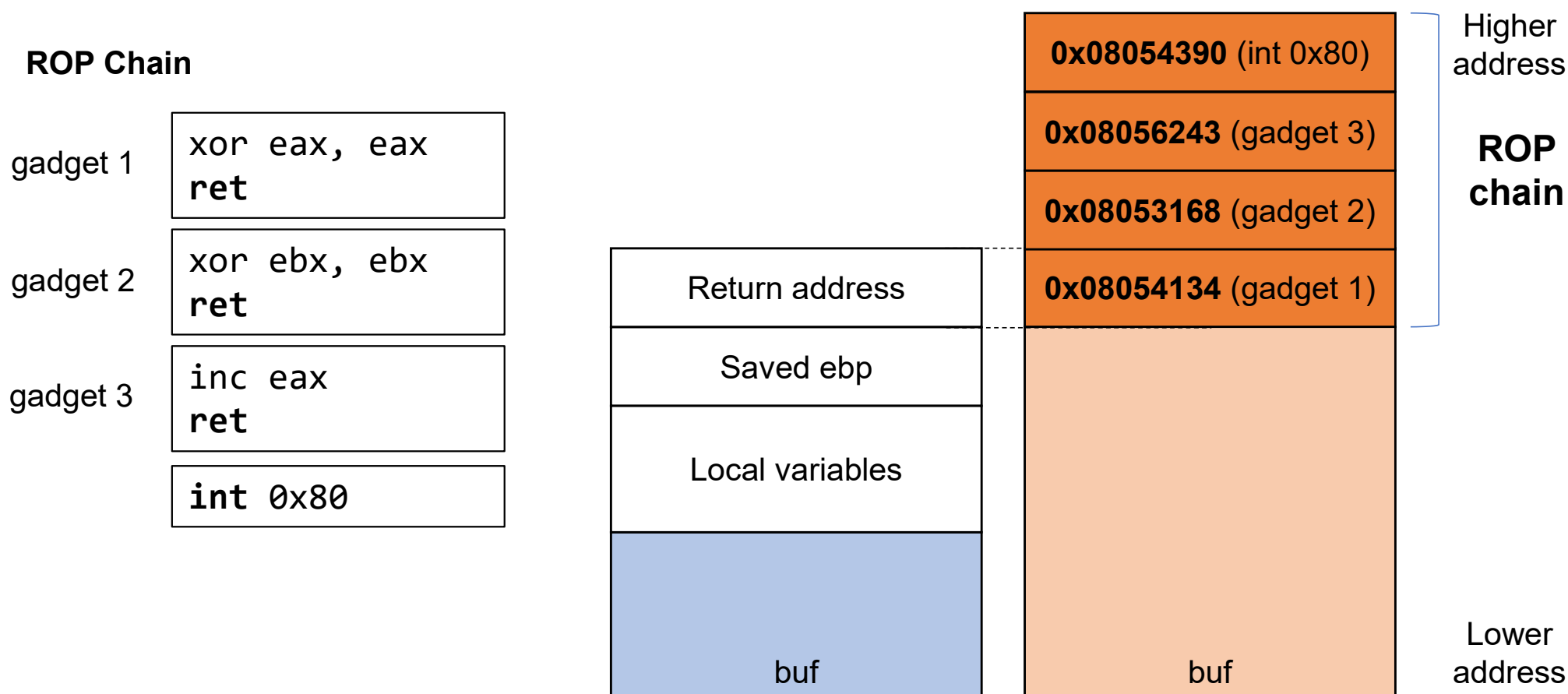
how to chain?

Tấn công ROP: ROP chain (2)



• ROP chain hoạt động như thế nào?

- Chain bằng cách sắp xếp các address của chúng ở các vị trí thích hợp trong stack.
- Address của gadget 1 sẽ ghi đè lên return address của chương trình.
- Address của gadget thực thi sau sẽ là return address của gadget thực thi trước.

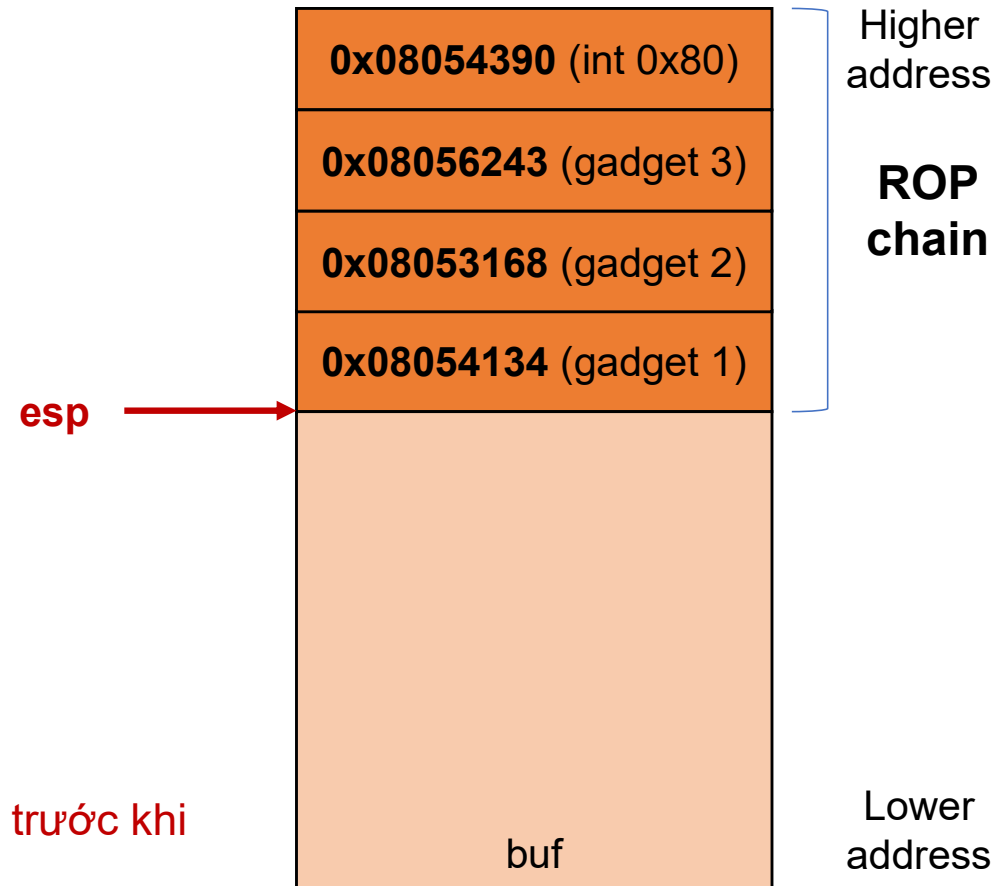


Tấn công ROP: ROP chain (3)

- ROP chain hoạt động như thế nào?

ROP Chain

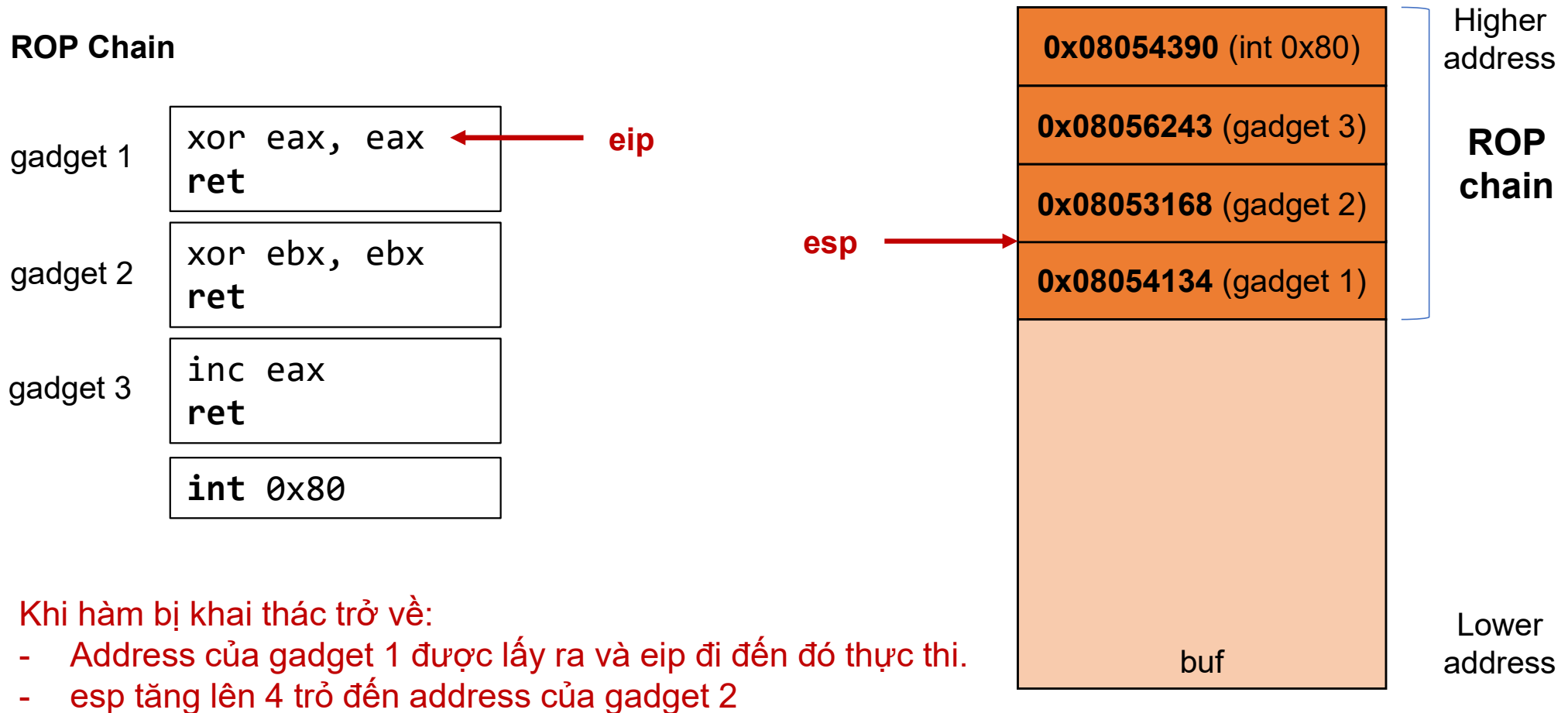
gadget 1	xor eax, eax ret
gadget 2	xor ebx, ebx ret
gadget 3	inc eax ret
	int 0x80



Sau khi nhập chuỗi **buf** để ghi đè return address và trước khi hàm bị khai thác trở về với lệnh **ret**.

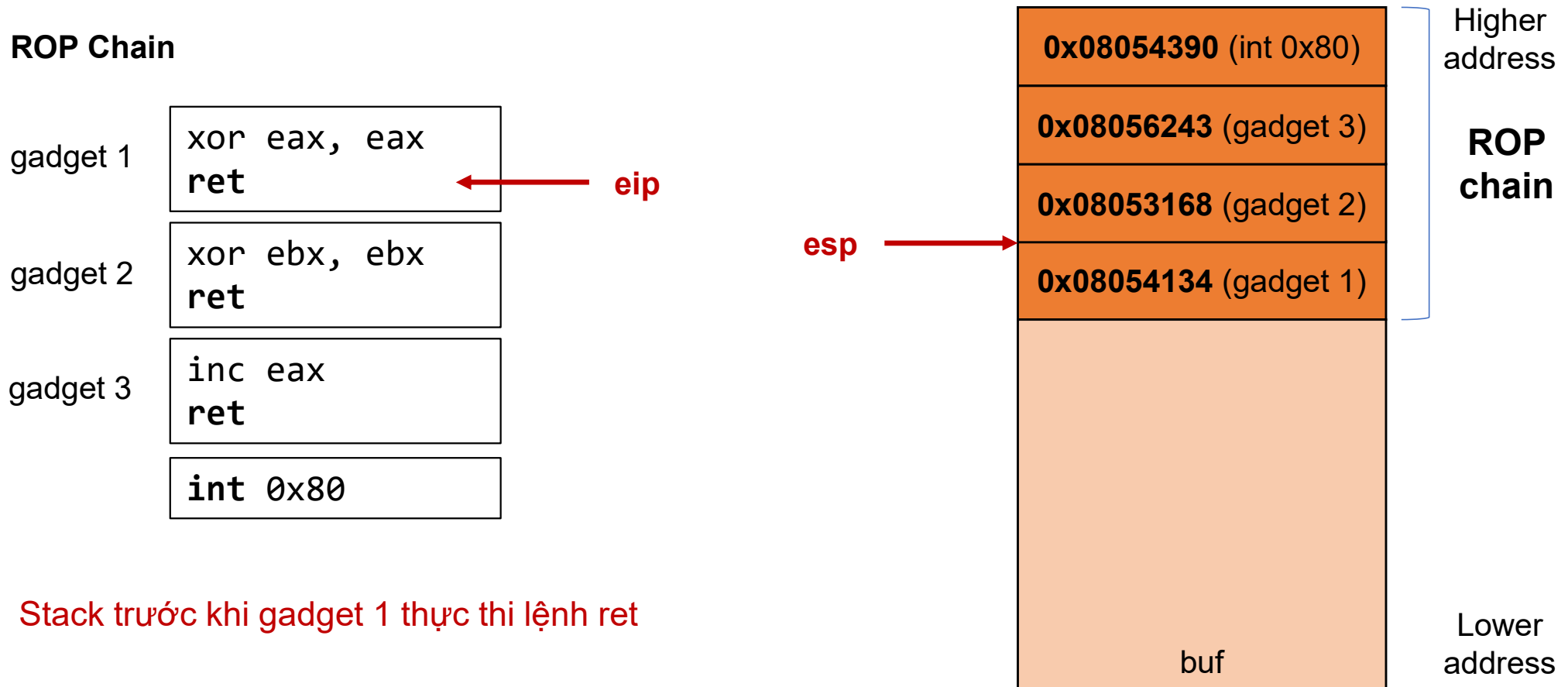
Tấn công ROP: ROP chain (4)

- ROP chain hoạt động như thế nào?



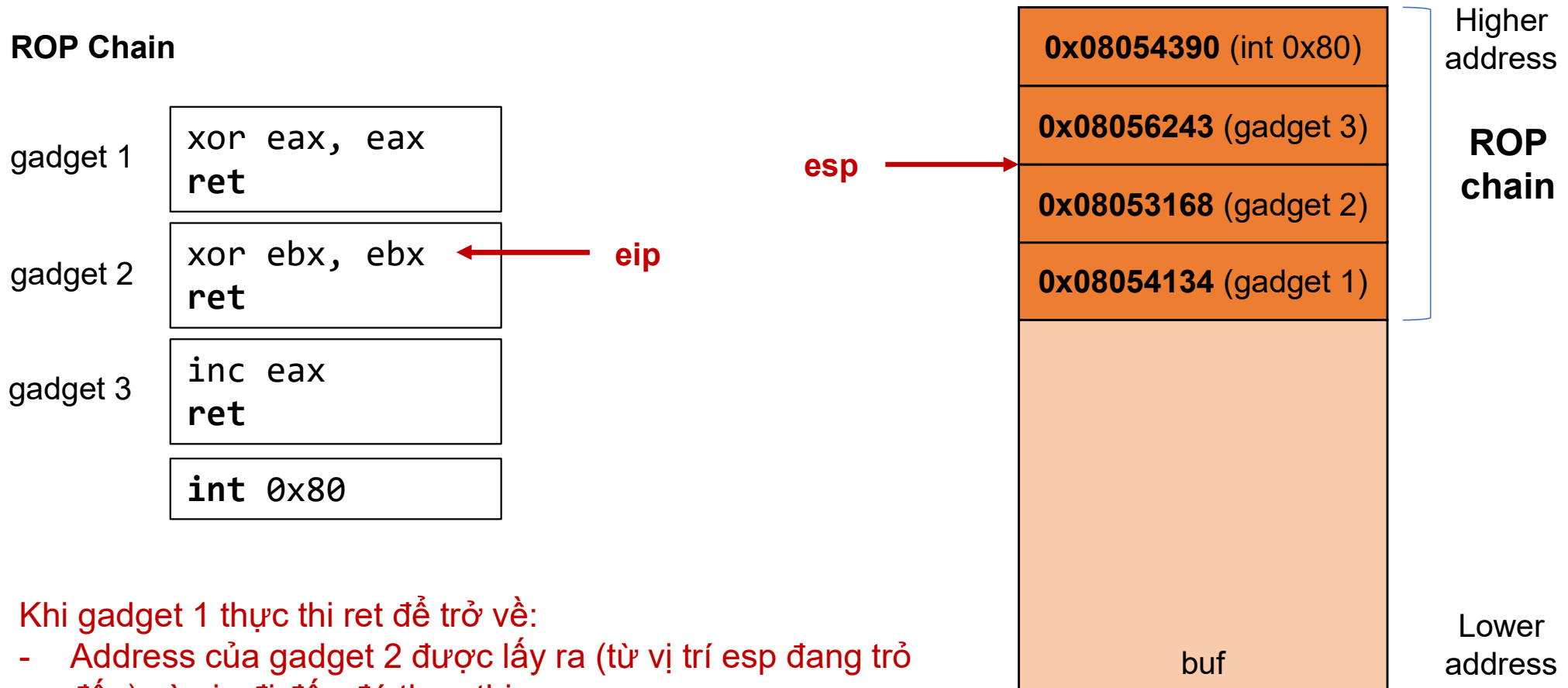
Tấn công ROP: ROP chain (4)

- ROP chain hoạt động như thế nào?



Tấn công ROP: ROP chain (5)

- ROP chain hoạt động như thế nào?

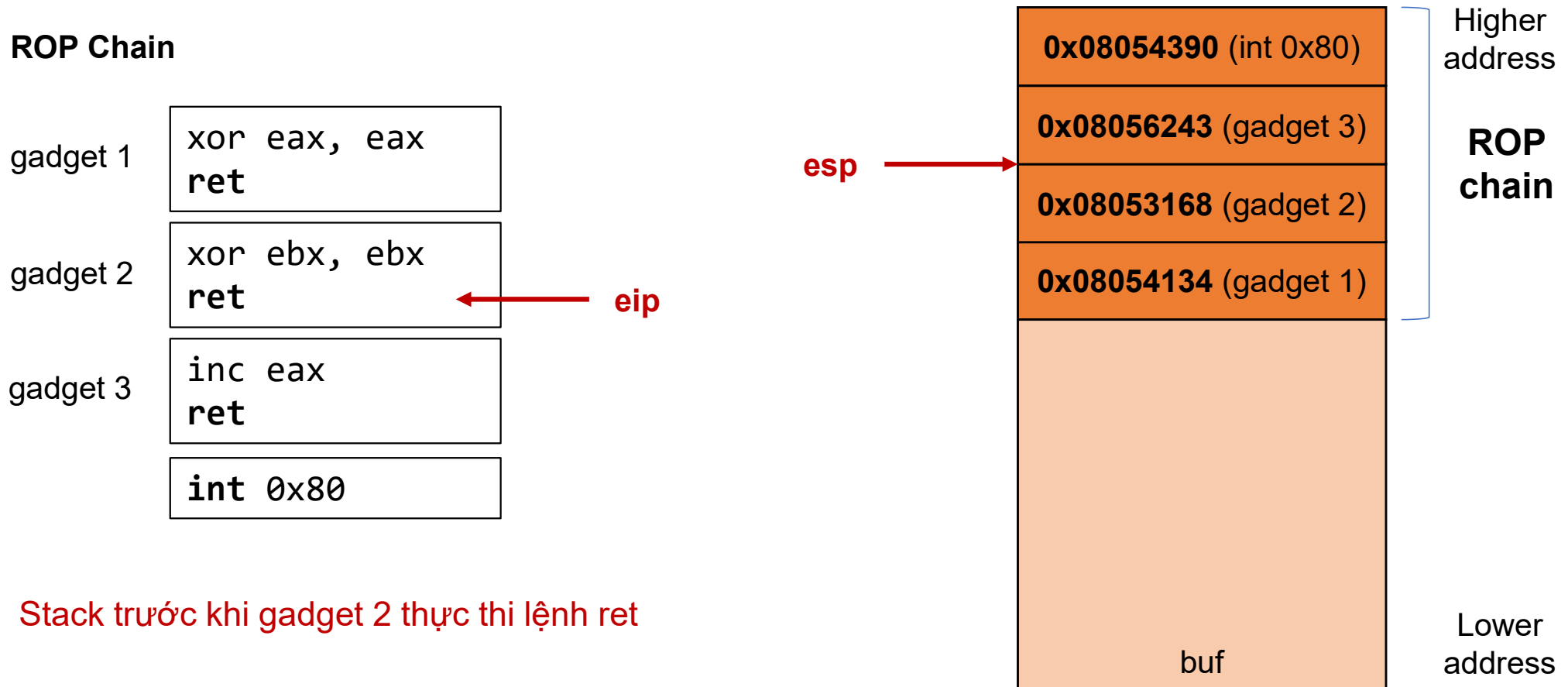


Khi gadget 1 thực thi ret để trở về:

- Address của gadget 2 được lấy ra (từ vị trí esp đang trỏ đến) và eip đi đến đó thực thi.
- esp tăng lên 4 trở đến address của gadget 3

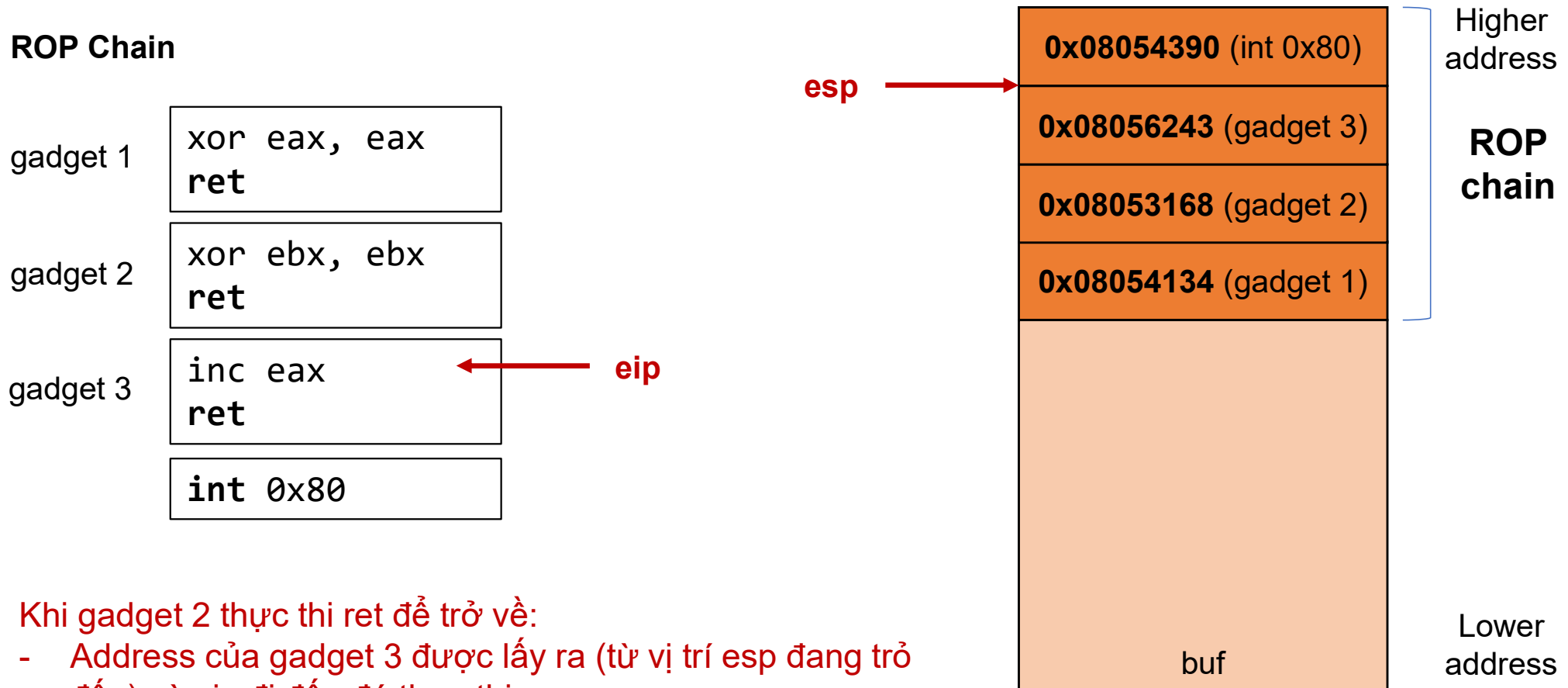
Tấn công ROP: ROP chain (6)

- ROP chain hoạt động như thế nào?



Tấn công ROP: ROP chain (7)

- ROP chain hoạt động như thế nào?

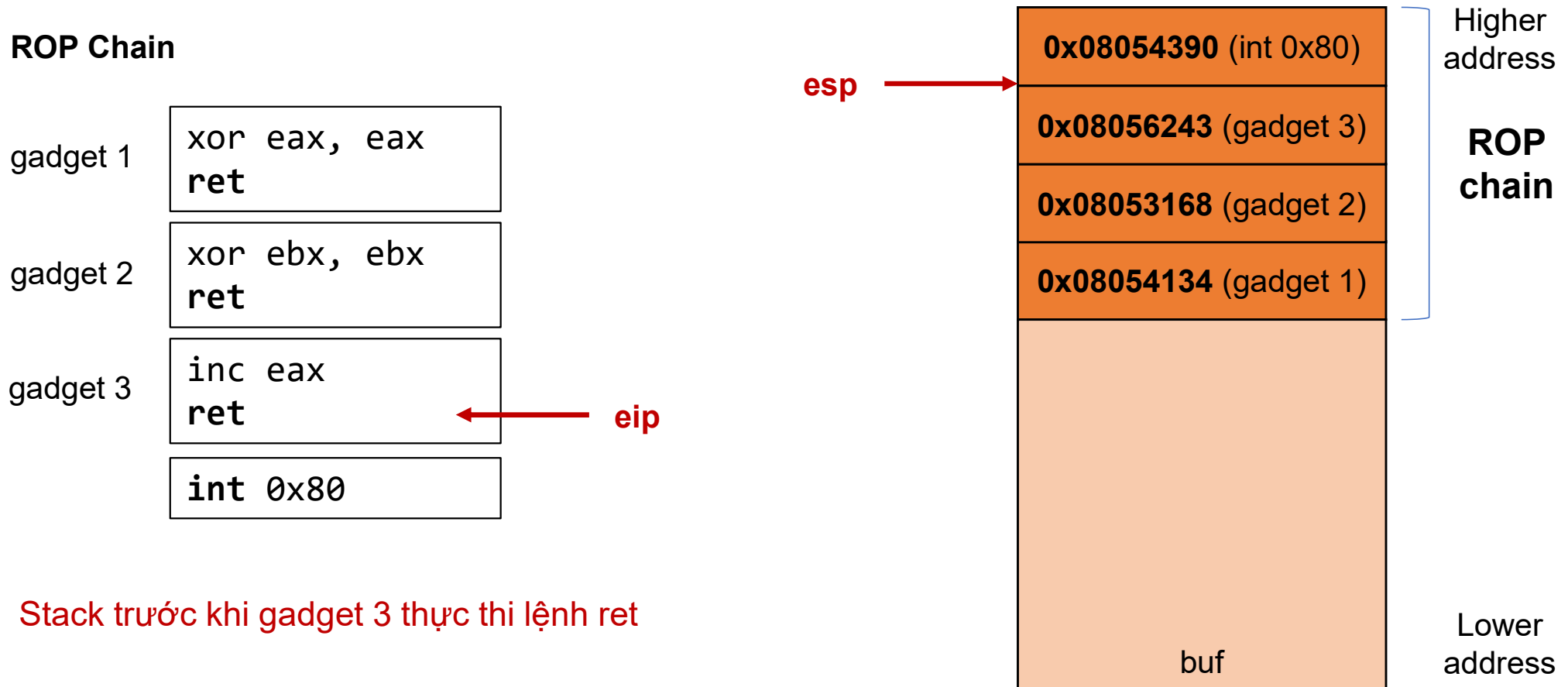


Khi gadget 2 thực thi ret để trở về:

- Address của gadget 3 được lấy ra (từ vị trí esp đang trở đến) và eip đi đến đó thực thi.
- esp tăng lên 4 trở đến address của lệnh **int 0x80**

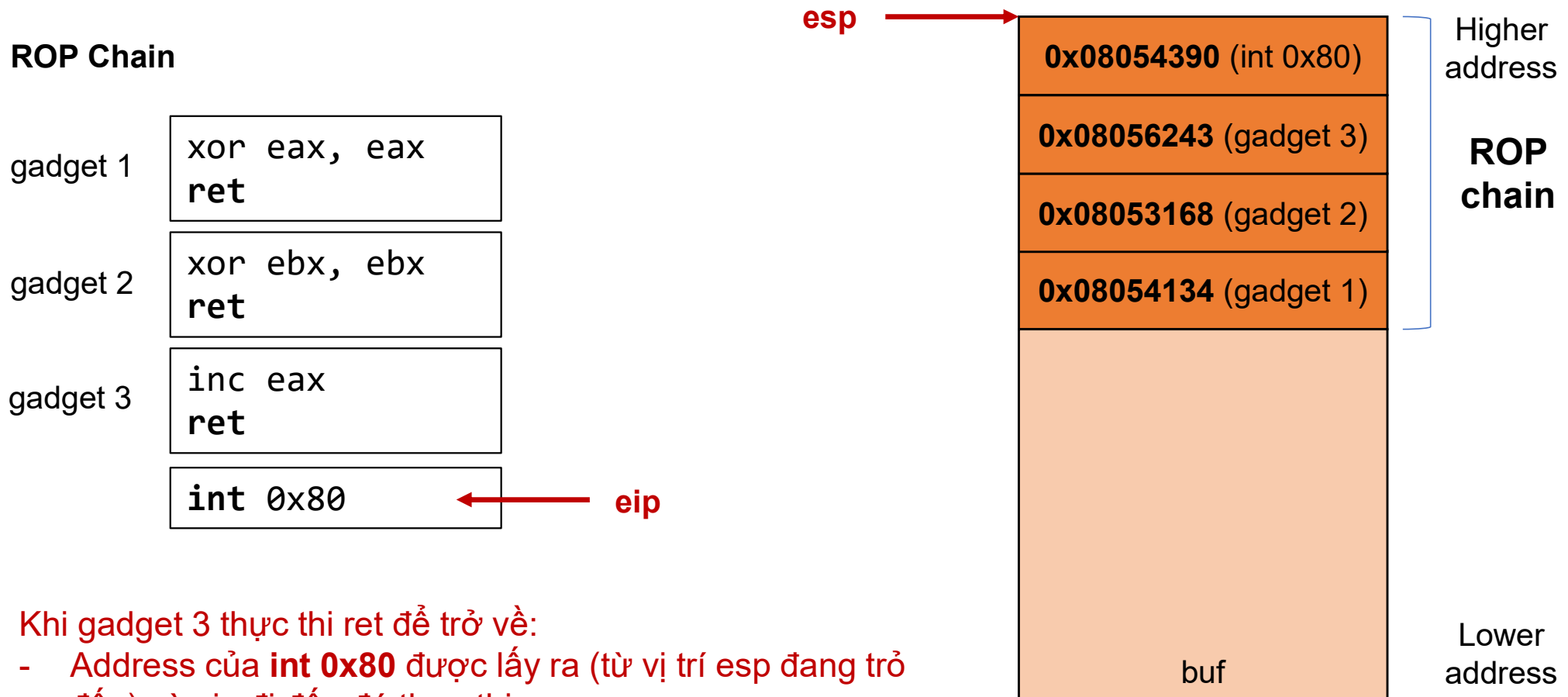
Tấn công ROP: ROP chain (8)

- ROP chain hoạt động như thế nào?



Tấn công ROP: ROP chain (9)

- ROP chain hoạt động như thế nào?



Khi gadget 3 thực thi ret để trở về:

- Address của **int 0x80** được lấy ra (từ vị trí esp đang trở đến) và eip đi đến đó thực thi.
- esp tăng lên 4 trở đến vị trí khác.

Tấn công ROP: Ví dụ 2



- Shellcode cần thực hiện

```
mov eax, 4          # eax = 4 (sys_write)
mov ebx, 1          # ebx = 1 (stdout)
mov ecx, str_addr   # ecx = address of string to print
mov edx, 13         # edx = length of string to print
int 0x80            # system call
```

- Các gadget tìm được

```
pop eax
ret
```

```
lea ecx, [esp+12]
ret
```

```
int 0x80
ret
```

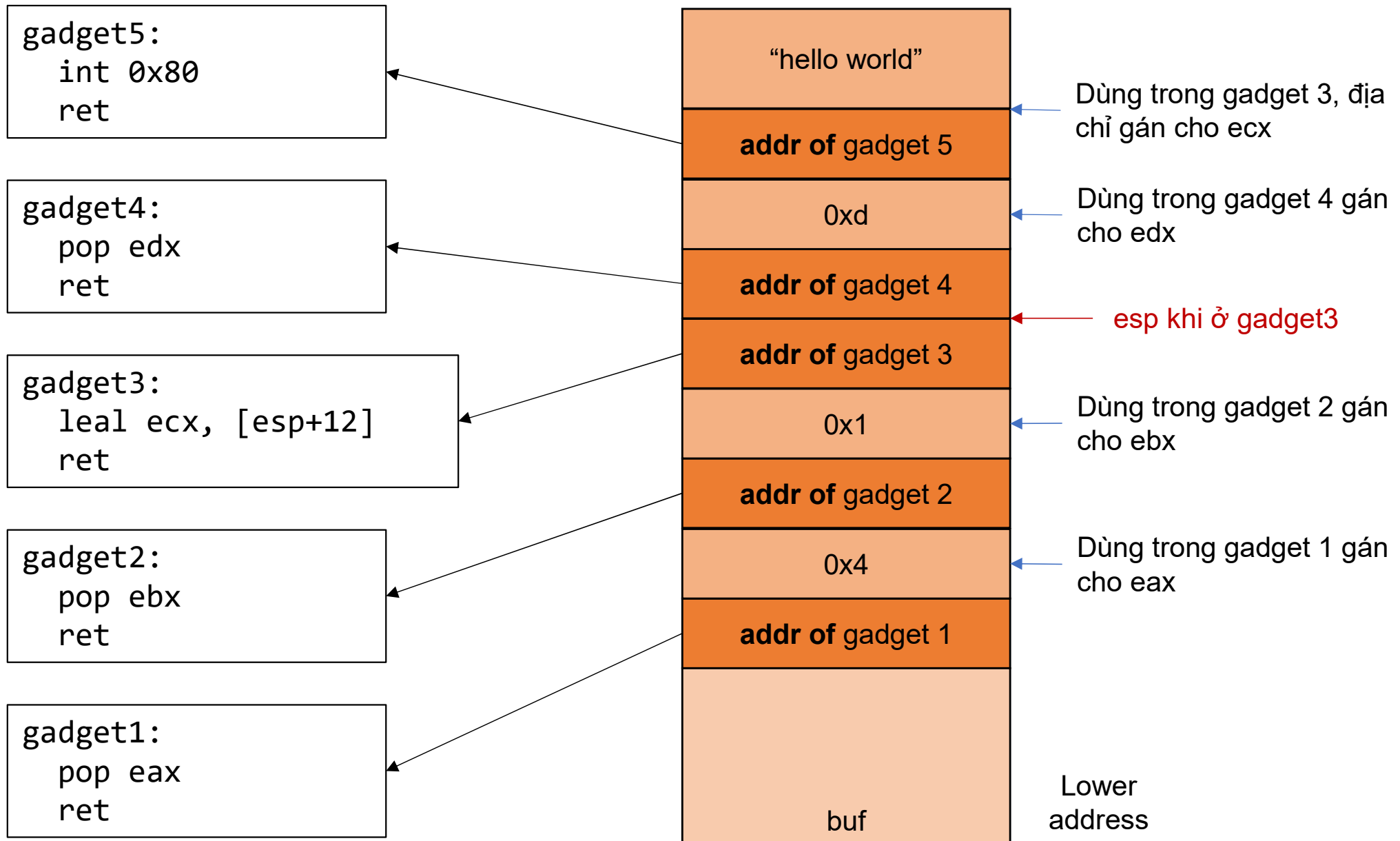
```
pop ebx
ret
```

```
pop edx
ret
```

- Có thể không tìm thấy các gadget để trực tiếp gán các giá trị như 4 hay 13 cho các thanh ghi
 - Các gadget trên lấy 1 giá trị nào đó trong stack gán cho các thanh ghi
- **Có thể đặt sẵn các giá trị cần gán trong stack để gadget dùng.**



Tấn công ROP: Ví dụ 2



Tấn công ROP: Stack pivoting



- *Sinh viên tự tìm hiểu Stack pivoting*



Format strings & ROP



- Sinh viên tham khảo thêm các slide trình bày miniseminar về các chủ đề liên quan.



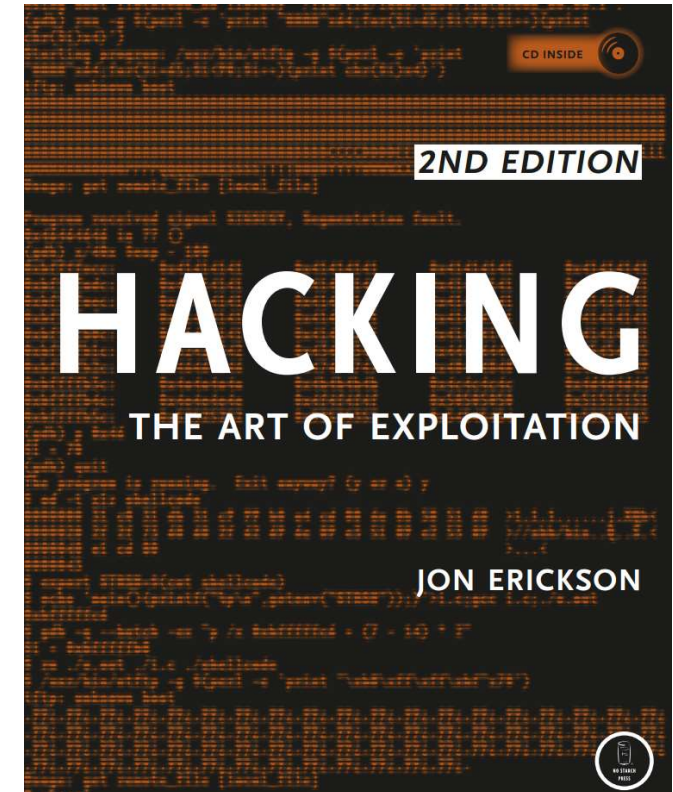
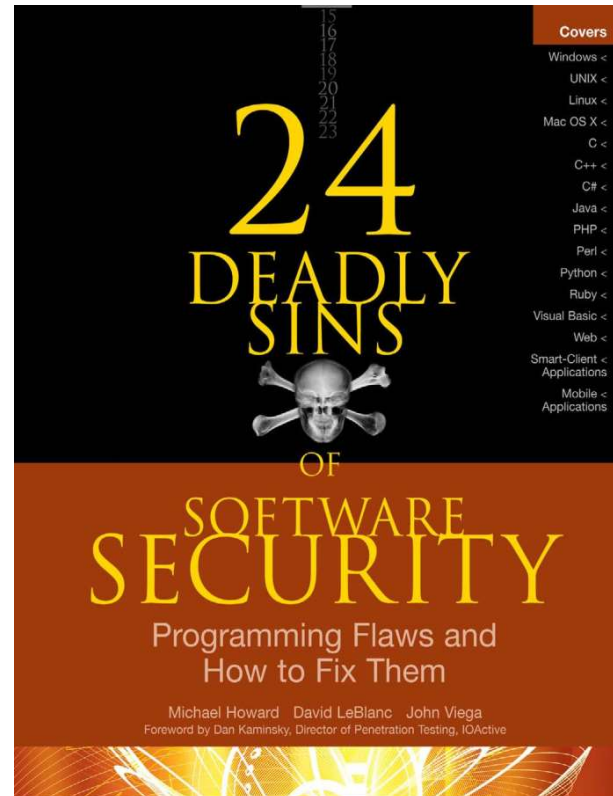
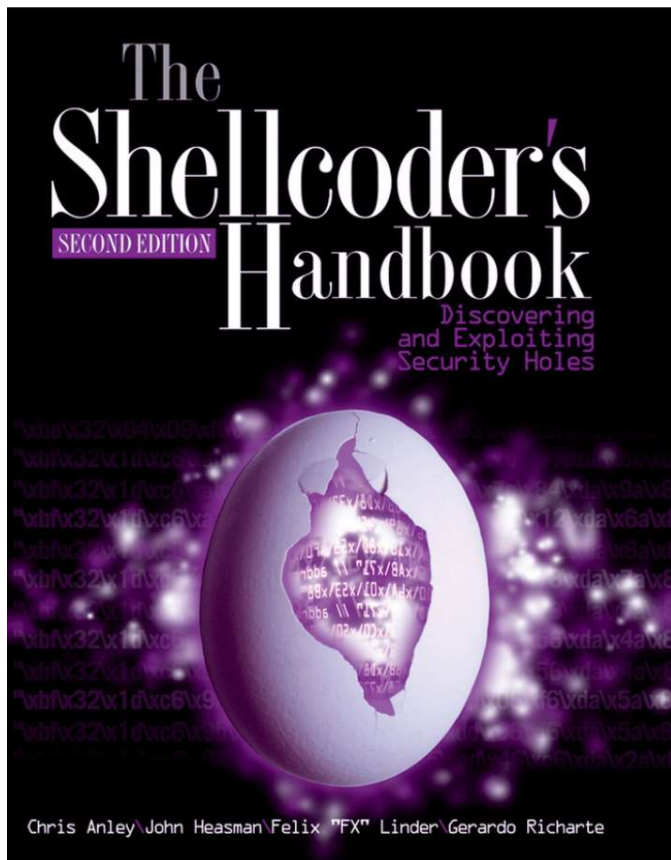
Tài liệu tham khảo: Pwn CTF

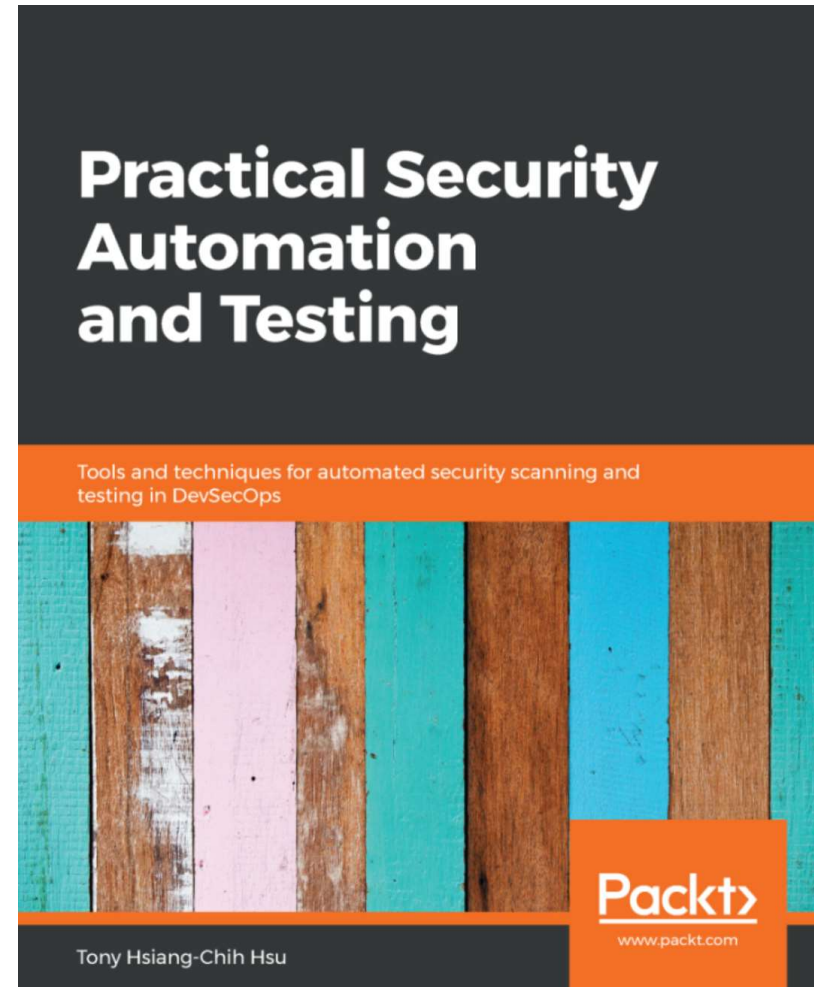
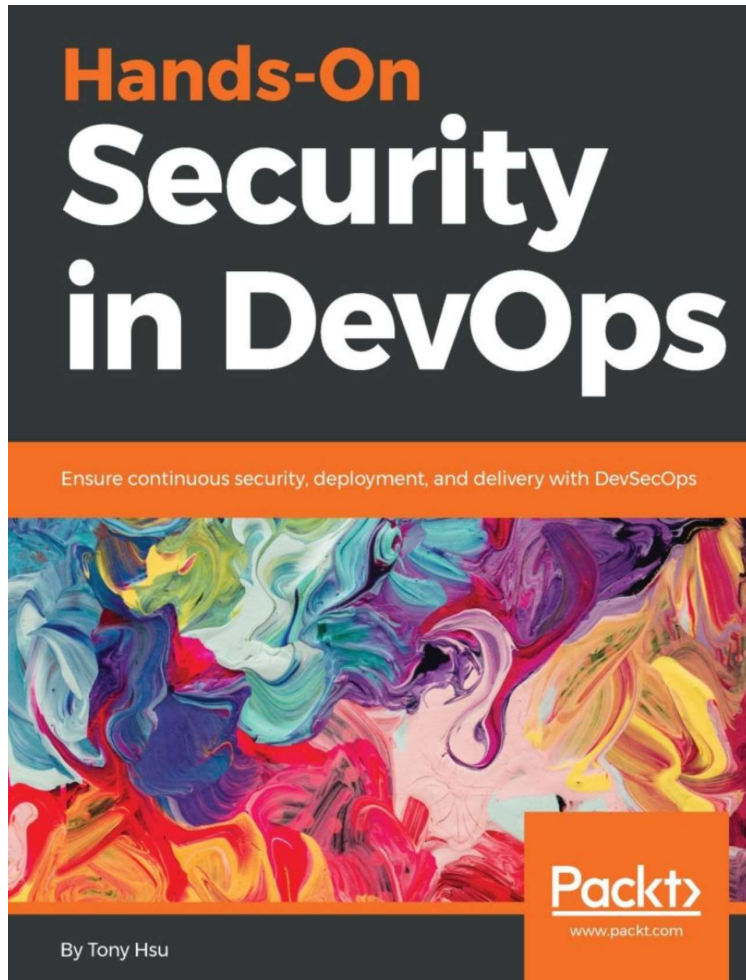


- CTF Wiki: <https://ctf-wiki.org/pwn/linux/user-mode/environment/>
- Modern Binary Exploitation - CSCI 4968: <https://github.com/RPISEC/MBE>
- NTU Computer Security Fall 2019: <https://github.com/yuawn/NTU-Computer-Security>
- Nightmare: <https://guyinatuxedo.github.io/index.html>



Tài liệu tham khảo





Tài liệu tham khảo



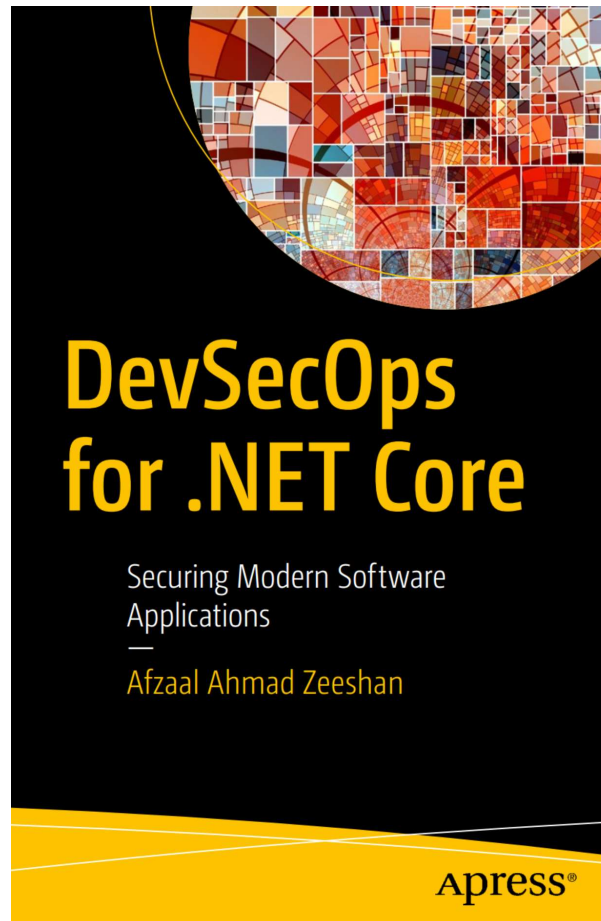
O'REILLY®



Agile Application Security

ENABLING SECURITY IN A CONTINUOUS DELIVERY PIPELINE

Laura Bell, Michael Brunton-Spall,
Rich Smith & Jim Bird



O'REILLY®

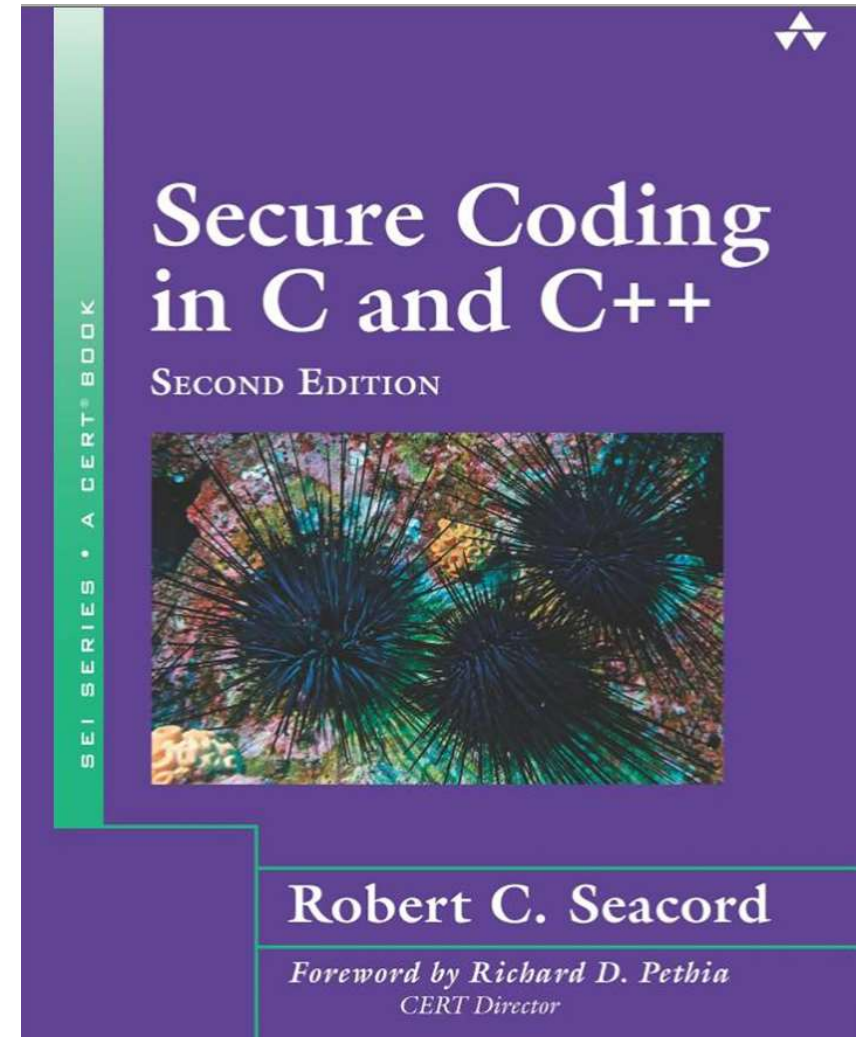
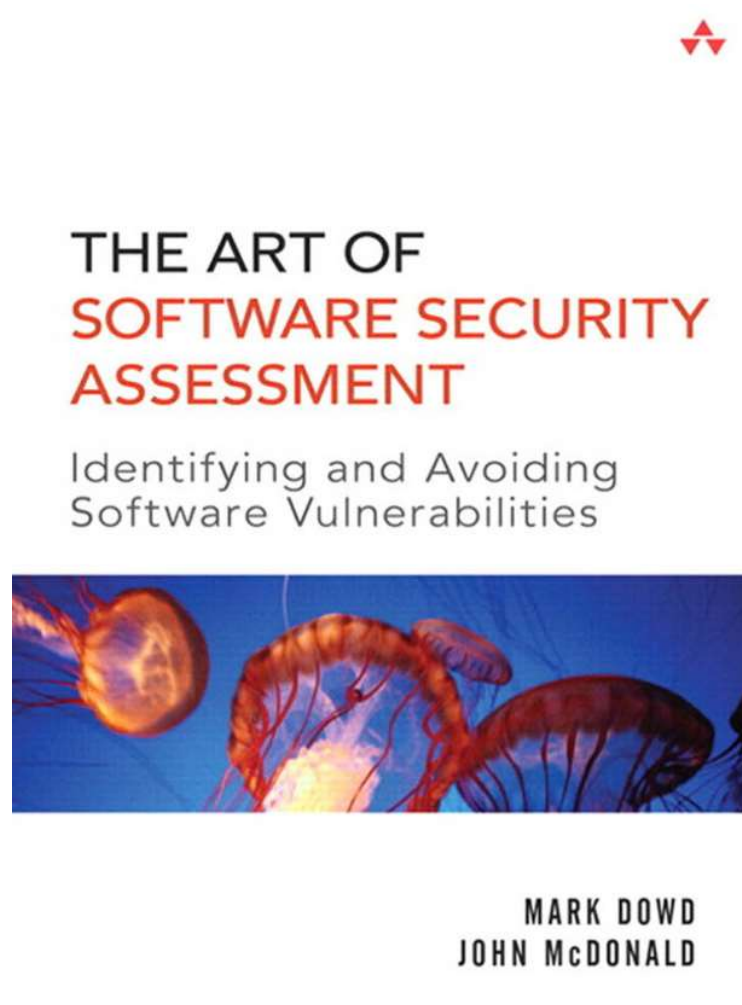
DevOpsSec

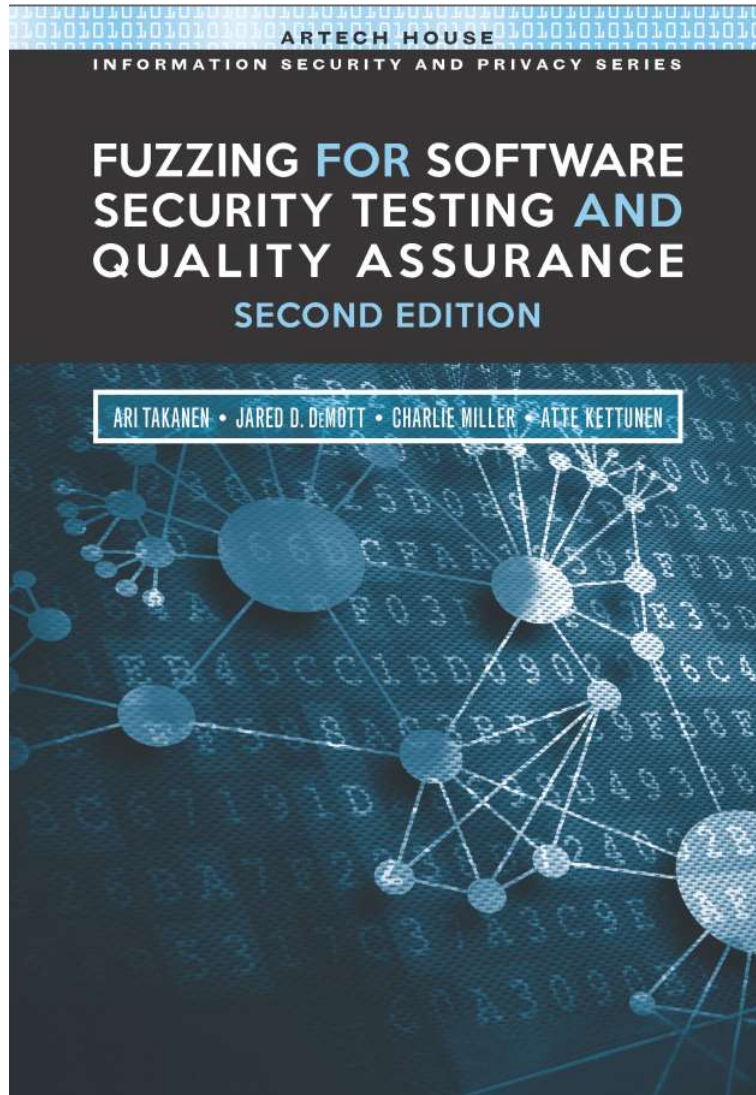
Delivering Secure Software
Through Continuous Delivery

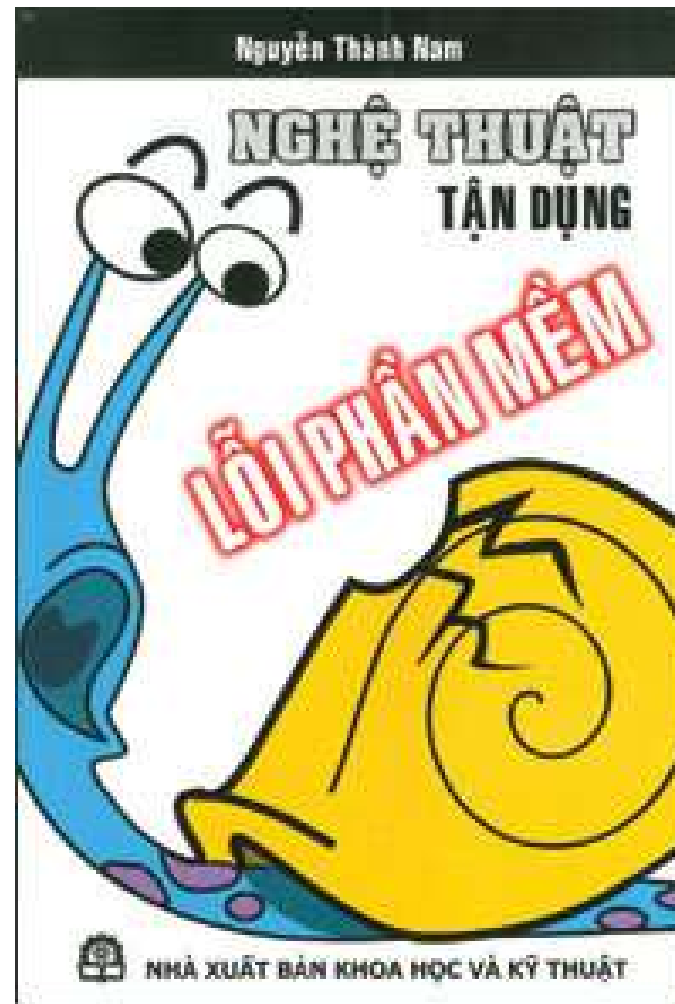


Jim Bird









- <https://security.berkeley.edu/secure-coding-practice-guidelines>
- <https://wiki.sei.cmu.edu/confluence/display/sec+code/Top+10+Secure+Coding+Practices>
- https://owasp.org/www-pdf-archive/OWASP_SCP_Quick_Reference_Guide_v2.pdf
- <https://www.softwaretestinghelp.com/guidelines-for-secure-coding/>
- <http://security.cs.rpi.edu/courses/binexp-spring2015/>
- <https://www.ired.team/>

Lập trình an toàn & Khai thác lỗi hỏng phần mềm



Trường ĐH CNTT TP. HCM